

Class 15: Model Monitoring and Drift Response

Data-driven Systems Engineering

Agenda

- Why monitoring matters
- Monitoring as risk control
- Monitoring types
- Data drift versus concept drift
- Detection strategies
- Performance over time and business metrics
- Response after alerts

Monitoring turns deployment into a feedback loop

Learning goals

- Distinguish system, data, model, and business monitoring layers.
- Compare data drift, concept drift, and pipeline failure symptoms.
- Design practical responses after a drift alert.
- Explain why monitoring starts before deployment rather than after it.

Course Map and Running Cases



Today in the course

Focus: Monitoring, drift response, and system evolution.

Bridge: This final class closes the lifecycle by showing how deployed systems remain trustworthy after release and how change feeds back into new work.

Fraud case

In the fraud case, monitoring covers data shift, score behavior, analyst workload, and the cost of false alerts over time.

Pasteurization case

In the pasteurization case, monitoring covers sensor quality, state transitions, prediction error, and when a drift alert should trigger investigation or retraining.

Course thread: The course ends where real systems live: not at deployment, but in continuous observation, response, and evolution.

Why monitoring matters

- ML systems are data products, not static binaries.
- Degradation can be silent and expensive.
- Labels may arrive late, so proxies matter.
- Monitoring is evidence for operations, governance, and improvement.

Monitoring is quantified risk management

- Data risk: changed distributions, missing features, schema violations.
- Model risk: degraded performance, instability, concept drift, unfairness.
- Operational risk: broken pipelines, latency spikes, failed endpoints.
- Business risk: review overload, poor intervention yield, loss of trust.

Meaning

Monitoring turns hidden failure into visible evidence that teams can act on.

What should be monitored in an ML service?

System layer: latency, errors, uptime, resources

Data layer: schema, ranges, drift, missingness

Model layer: scores, errors, calibration

Business layer: review load, outcomes, overrides

Classical monitoring versus ML monitoring

Traditional systems

- CPU, memory, disk
- Request rate and latency
- Availability and error codes

ML systems add

- Feature quality and schema checks
- Drift and score shifts
- Delayed labels and performance decay

Data drift vs concept drift

- Data drift: input distribution changes, even if the relationship still holds.
- Concept drift: the mapping from input to target changes.
- Pipeline bugs can mimic both and must be ruled out early.

Examples

Holiday shopping patterns can cause fraud feature drift; a new attack strategy can create concept drift.

What can look like drift but is not

- A broken ingestion job that changes missing-value rates.
- A schema change that reorders or renames columns.
- A deployment mismatch between training preprocessing and live preprocessing.
- A monitoring bug that changes the way metrics are computed.

Rule

Before retraining, verify that the problem is real and not a pipeline defect.

- Batch statistical tests: KS, PSI, chi-square, Jensen-Shannon variants.
- Prediction monitoring: score distribution and confidence shifts.
- Label-based metrics when ground truth arrives.
- Streaming detectors: ADWIN, Page-Hinkley, DDM, EDDM.

A practical drift-detection taxonomy

Signal type	Typical use
Feature distribution tests	Early warning when labels are delayed.
Prediction distribution checks	Detect score collapse or threshold instability.
Label-based performance	Confirm concept drift when outcomes arrive.
Streaming detectors	Catch online change in error or metric sequences.
Segment analysis	Reveal that drift affects only some populations.

Repository-style online monitoring loop

```
y_pred = model.predict_one(x)
model.learn_one(x, y)
mae.update(y, y_pred)
error = abs(y - (y_pred or 0))
adwin.update(error)
drift_flag = adwin.drift_detected
```

- 'streamlit/pages/App_Stream.py' uses this pattern to combine incremental learning, error tracking, and ADWIN-based change detection.
- It is useful because the alert is tied to live behavior, not only offline charts.

Repo examples for a stronger lecture

Repo activity

- 'streamlit/pages/App_Stream.py' already combines online learning with ADWIN.
- 'documents/Forecasting_and_Drift_Pasteurization.pdf' defines a project around forecasting and drift detection.
- 'pasteurization/synth_sensors.py' gives state-aware signals for meaningful monitoring discussions.

Labels arrive late, so proxies become important

- In many real systems, ground truth appears hours, days, or weeks later.
- Teams therefore monitor proxies such as score distribution, alert rate, abstentions, or operator overrides.
- Proxy metrics are not substitutes for truth, but they reduce blind spots between retraining cycles.

What to do after detection

1. Verify the alert is not caused by ingestion or schema issues.
2. Diagnose whether the change is temporary, seasonal, or structural.
3. Choose a response: retrain, recalibrate, switch model, or escalate to humans.
4. Record the event for later audit and process learning.

Typical responses after a monitoring event

- Retrain when the environment has genuinely shifted and new labels are available.
- Recalibrate when confidence is wrong but ranking remains useful.
- Roll back when a release introduced the failure.
- Add human review when uncertainty is high and automated action is risky.
- Change thresholds when business cost trade-offs have shifted.

Performance over time and business metrics

- Model metrics: accuracy, precision, recall, AUC, MAE, RMSE.
- Operational metrics: latency, queue length, failed predictions.
- Business metrics: intervention yield, review load, false alert fatigue.
- Human signals: overrides, complaints, and trust breakdowns.

Champion-challenger and continuous comparison

- A champion model serves production traffic.
- A challenger runs in shadow mode or on sampled traffic.
- Comparison over time helps avoid switching on the basis of one lucky offline result.
- This strategy is especially useful when concept drift is suspected but labels are delayed.

From This Class to the Next

What stays with us

- Monitoring turns deployment into an ongoing feedback process instead of an endpoint.
- Drift detection matters only when the team has diagnosis and response mechanisms.
- The full course lifecycle closes here: systems must evolve as data, users, and risks change.

Project use

For both running cases, long-term value depends on observing data quality, model behavior, operational health, and human consequences over time.

Next connection

This closing class links back to the first one: every architecture, feature, model, interface, and process choice should support sustainable evolution.

Course thread: The course ends with the same systems view it began with, now grounded in concrete engineering practices across the whole lifecycle.

Papers and materials

River ADWIN documentation; Practical MLOps; Reliable Machine Learning; Gama et al., A survey on concept drift adaptation; Evidently on data drift detection; Prometheus monitoring overview.